

# A Data Structure for Managing Bitcoin's UTXO Set

Peter R. Rizun<sup>†</sup>

**Abstract.** Managing bitcoin's UTXO set in a way that permits significant growth while retaining low-cost open access is important. Global adoption would not be feasible without a scalable solution but the decentralized and permissionless nature of the bitcoin network would be threatened if access to the UTXO set were too costly. We propose a simple data structure for managing bitcoin's UTXO set that is parallelizable, persistent, and authenticated by proof-of-work. Horizontally scaling the UTXO database in hardware would become straightforward, while parallel validation of competing blocks and instant blockchain reorganizations would be natively supported. Furthermore, new network nodes could be bootstrapped by downloading the UTXO set from an untrusted source rather than by downloading and rebuilding the set from the much-larger blockchain. Lastly, users could verify unconfirmed bitcoin transactions without maintaining the UTXO set themselves by using a compact proof received from a network node instead. On Github we provide two example implementations for manipulating the data structure: software written in Python, and a hardware circuit described in Verilog suitable for deployment on an ASIC or synthesis on a FPGA.

## 1. Introduction

On the Bitcoin network, a *coin* is simply an output of a previous transaction. The collection of every spendable coin that exists is known as the *unspent transaction output* (UTXO) set. Each network node maintains a copy of the UTXO set in order to verify new transactions. But maintaining the UTXO set is not trivial.<sup>1</sup> More users means that bitcoin's fixed money supply is split across a greater number of transaction outputs—amounting to potentially hundreds of billions of coins if bitcoin were adopted globally. Dealing with a large number of coins is not the only obstacle: to keep the UTXO set up-to-date, nodes must listen for and process every new transaction submitted on the network—potentially hundreds of thousands of transactions per second at scale. Nodes that mine for new bitcoins are further burdened by maintaining multiple *views* of the UTXO set to validate competing blocks in parallel and efficiently deal with blockchain reorganizations. Also, there is no secure way for a new node to acquire the current UTXO set other than by downloading and replaying every historical bitcoin transaction since the genesis block—hundreds of gigabytes of data today and potentially exabytes in the future. Bitcoin users are impacted too: maintaining the complete UTXO set is not realistic with a mobile wallet. A user cannot independently verify transactions when first received and must trust a third party or wait until the transaction is confirmed in a block, limiting bitcoin's usefulness as a payment system. Managing the UTXO set in a way that allows for future growth while still permitting low-cost access by new network nodes and regular users is a challenge.

This paper proposes a data structure for managing bitcoin's UTXO set to address this challenge. Most importantly, the structure is *parallelizable*, *persistent*, and *authenticated*. By *parallelizable* we mean that the structure can easily be split up across independently-managed cells that operate concurrently. This property simplifies horizontal scaling of the database used

---

<sup>†</sup>P. Rizun, Ph.D. (peter.rizun@gmail.com) is a physicist and entrepreneur from Vancouver, Canada and Chief Scientist for Bitcoin Unlimited.

to store the UTXO set. By *persistent* we mean that updates to the UTXO set are made in a way that leaves historical views of the set intact. This property enables native parallel validation of competing chain tips and instant blockchain reorgs, both of which help scale mining nodes. By *authenticated* we mean that a user can determine whether a particular coin was in the UTXO set at a given point in time from a compact proof received from an *untrusted* source. This property makes it possible for users' mobile wallets to validate unconfirmed bitcoin transactions, as well as for new network nodes to be launched by downloading the UTXO set directly (3.6 GB today) rather than rebuilding it from the blockchain (250 GB today). A positive side effect of the authentication process is that it becomes impractical for a mining node to deviate from the protocol<sup>2</sup> to mine a new (empty) block before downloading and processing the previous block.

## 2. Binning by Binary Prefix

To split the job of managing the UTXO set between parallel processes, we first define a deterministic way to place the outputs into bins. Since a bitcoin transaction spends an output by specifying its `txid` and `index`, a hash taken over this information can serve as a *key*.<sup>3</sup> The *value* that needs to be retrieved (to verify that the output can be spent) includes its `scriptpubkey`, its amount in satoshis, and a few more details.<sup>4</sup> This value can be stored alongside the key in the appropriate bin.

We bin the outputs according to the *binary prefixes* of their keys. In the example shown in Fig. 1, outputs with keys with binary prefix `00**` are stored in the first bin, prefix `01**` in second, `10**` in the third, and `11**` in the last. The outputs in each bin can then be managed independently, and located on different pieces of physical hardware. We will refer to each independently-managed bin as a *cell*.

| PREFIX: 00**                                             | PREFIX: 01**      | PREFIX: 10**                                                | PREFIX: 11**                         |
|----------------------------------------------------------|-------------------|-------------------------------------------------------------|--------------------------------------|
| 0000: (Bob, 9฿)<br>0011: (Iris, 6฿)<br>0001: (David, 1฿) | 0111: (Henry, 8฿) | 1001: (Alice, 2฿)<br>1011: (Frank, 7฿)<br>1000: (Carol, 3฿) | 1111: (Ella, 2฿)<br>1110: (Gina, 4฿) |

Fig. 1. The outputs are binned according to the binary prefix of their keys. Each bin can be managed independently from the other bins. The length of the key space in this example is 4 bits; much longer key spaces (e.g., 256 bits) would be used in practice.

Cells are oblivious to bitcoin transactions and are only concerned with carrying out `spend` and `mint` operations for outputs in their prefix range (`spend` deletes an output from the set while `mint` adds a new one). Concurrency control at the cell level is limited to ensuring only that the same coin cannot be spent twice; this is straightforward to ensure if `spend` and `mint` operations are performed serially within each cell. Individual bitcoin transactions must be committed *atomically*, of course, but there is no need for locks: when verifying a group of transactions, each transaction's inputs can be speculatively spent in any order. Should the spending of an input in a given transaction fail (e.g., because it was already spent by another

transaction in the group), then all the speculatively-spent inputs for the failed transaction must be “unspent” and the transaction rejected or moved to a retry queue.<sup>5</sup>

The database used to maintain the UTXO set can easily scale horizontally by adding more cells, in a manner that is nearly invisible to the other components of the bitcoin node.

### 3. A Nested and Deterministic Structure

The process of binning by prefix can be abstracted to create a data structure ideal for making both *persistent* (Section 5) and *authenticated* (Section 12). To begin, note that from the perspective of any one of the bins in Fig. 1, the problem of managing its subset of UTXOs is identical to the original problem of managing the global set—only for a smaller key space. Each bin can be further subdivided into smaller bins.

Taken to the limit, the result is the nested structure of bins shown in Fig. 2. Each bin contains two smaller bins that specify one more bit of the keys’ prefixes. These bins in turn each contain two even smaller bins that specify one further prefix bit still. This pattern continues until we get to a row of bins with prefixes so long that all but the least significant bit (LSB) is specified. Into one of these bins, only two outputs can be placed: an output with a key that matches the bin’s prefix and has a LSB of 0 (left side of bin), and an output with a key that matches the bin’s prefix and has a LSB of 1 (right side of bin). Since there is exactly one possible position in the structure that an output with a given *key* can be placed, the structure is deterministic—its shape does not depend on the order in which the outputs were added.

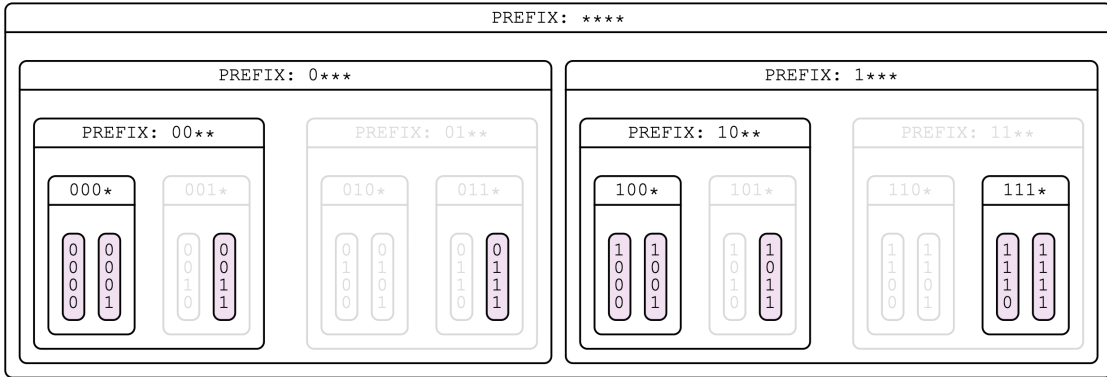


Fig. 2. Binning by the binary prefix of the keys can be extended, resulting in a nested structure of bins, shown here for a four-bit key space. Each bin in the last row can only hold two outputs—one with a key that matches the bin’s prefix with a least significant bit of 0 and the other with a key that matches the bin’s prefix with a least significant bit of 1. Removing all bins with fewer than two items (those that are grayed out) results in a structure that can be physically constructed for large key spaces. Exactly  $N - 1$  bins are required, where  $N$  is the number of stored elements.

This structure is not physically realizable for large key spaces because the number of bins required is equal to cardinality of the key space minus one (e.g., a 256-bit key space would require  $2^{256} - 1$  bins). We can reduce the number of bins to  $N - 1$ , where  $N$  is the number of

stored elements, by dissolving every bin that contains fewer than two items (the dissolved bins are shown grayed out in the Fig. 2). The structure is now *compact*.

#### 4. Compact Bitwise Prefix Trie

We will refer to the data structure constructed in Section 3 as a *compact bitwise prefix trie*. The word *trie* comes from *retrieval* and is pronounced ‘tree’ to reflect its shape. It can be visualized in tree form as shown in Fig. 3. The first bin becomes the trie’s *root*, the transaction outputs become the *leaves*, and the inner bins become the *interior nodes*. A path from root to leaf through a sequence of interior nodes is called a *branch*.

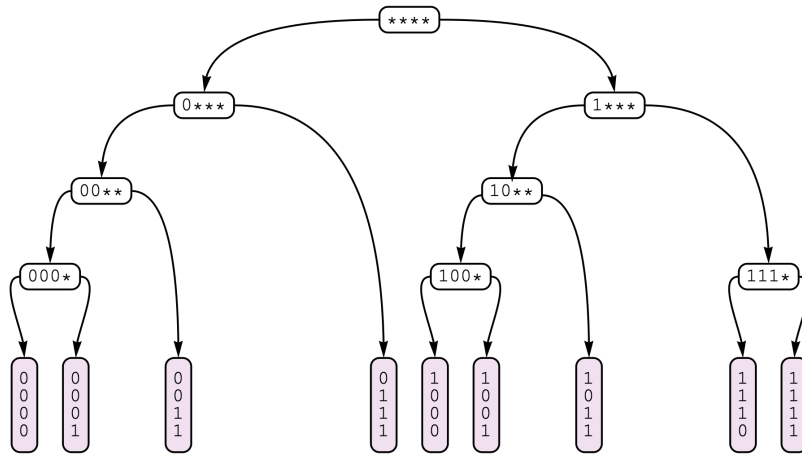


Fig. 3. The UTXO set is stored in memory as a bitwise prefix trie. The horizontal ovals are interior nodes, and point to right and left subtrees and describe the binary prefix common to elements in both. The purple boxes contain the output’s scriptpubkey and amount in satoshis (not shown) and can be keyed, for example, with a hash taken over the transaction ID and output index.

Navigating the trie to look up an output by key has  $O(\log N)$  complexity,<sup>6</sup> where  $N$  is the number of leaves. If the trie is balanced,  $\log_2 N$  interior nodes must be traversed. If the keys are randomly distributed across the key space instead, some branches will be longer than  $\log_2 N$  and some will be shorter, but none will have a length greater than the number of bits in the key space,  $K$ . Search latency thus increases slowly versus the number of elements in the trie (Section 10 discusses skipping over the base of the trie to expedite look-ups).

#### 5. Persistent Data Structure

The blockchain is often considered an immutable record of transactions; however, the tip of the chain is an uncertain place. At any given time, there may be multiple tips, possibly with conflicting transactions, competing to outgrow the others and secure a permanent spot in the chain. The miners work upon these chain tips and every millisecond counts for profitability. Each chain tip corresponds to a different *view* of the UTXO set, and dealing with these different views is sufficiently awkward that the Core implementation cannot validate competing blocks

in parallel.<sup>7</sup> Blockchain reorgs then become slow and potentially costly to miners. This lack of parallel validation is an impediment to scaling.

What we need is a structure that allows the bitcoin node to validate multiple chain tips simultaneously without bogging down the node with complex context switches and concurrency control schemes. Our trie can be made to support parallel validation *natively*, in a way that is surprisingly simple yet performant. To do this, we make the trie a *persistent data structure*.<sup>8</sup>

The critical observation is that any transaction output can be added or removed—in a manner that keeps the existing trie in place—by adding only a single new branch. Since the length of a trie branch scales with the logarithm of the set size, this proposition remains feasible for large sets. Fig. 4 illustrates adding an output with key 1100. The trie that is accessible from the new root contains the new output; however, the previous version of the trie (where 1100 does not exist) remains unchanged and accessible from the old root.

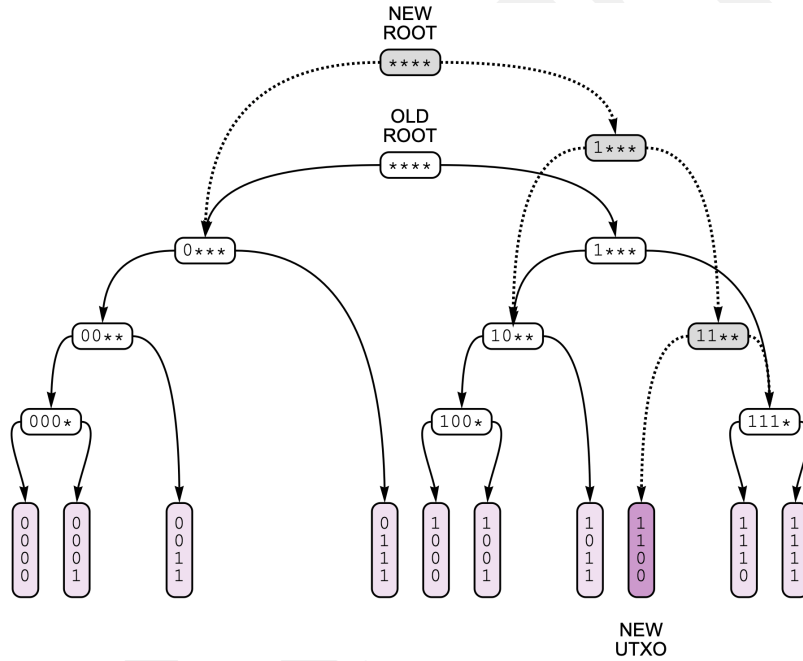


Fig. 4. Any output can be added or removed in a way that leaves the previous UTXO set state intact by adding a single new branch. In the illustration above, an output with key 1100 is added to the trie by adding the branch shaded in gray. The previous state of the set (before 1100 was added) persists and is accessible via the trie’s old root. Note that the keyed boxes shaded purple also contain the output’s `scriptpubkey` and amount in satoshis, but are not explicitly shown in the diagram.

## 6. Native Parallel Validation

Fig. 5 illustrates how a network node can process two competing blocks in parallel using the same instance of the persistent data structure. Assume that at time  $t_0$  the best chain tip corresponds to block  $X$  and the UTXO set consists of all the outputs accessible from root 0. At time  $t_1$ , the node learns of two new blocks: block  $Y$  that mints output 1100 and spends output 0011, and a competing block,  $Y'$ , at the same height, that also spends 0011 but mints 1101

instead. The purple nodes in the figure correspond to the updates made while processing  $Y$ . The purple branch at root 1 adds the output 1100 to the UTXO set defined by root 0, and then the purple branch at root 3 removes output 0011. Root 3 thus defines the UTXO set from the view of block  $Y$ . The blue nodes correspond to updates made while processing  $Y'$ , which occur in parallel to the updates made while processing  $Y$ . The blue branch at root 2 removes output 0011 from the UTXO set defined by root 0, and then the blue branch at root 4 adds output 1101. Root 4 defines the UTXO set from the view of the competing block  $Y'$ . Either chain tip ( $Y$  or  $Y'$ )—or both chain tips simultaneously—can be built upon without loss of efficiency, limited only by physical memory.

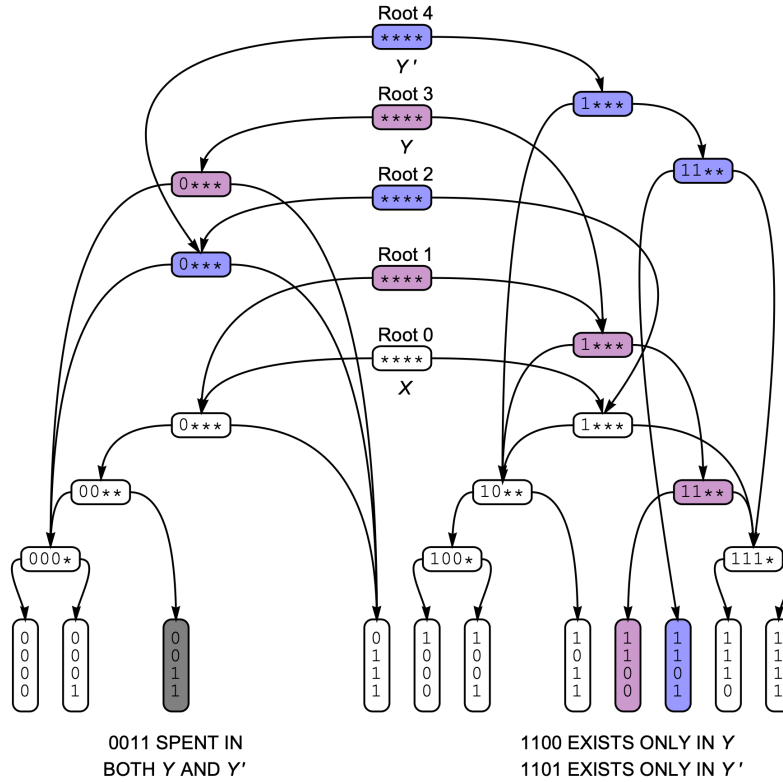


Fig. 5. Block  $Y$  and  $Y'$  can be processed in parallel against the same instance of the data structure. Block  $Y$  mints output 1100 and spends output 0011. Block  $Y'$  also spends output 0011 but mints output 1101 instead. After processing the blocks, UTXO views corresponding to two different chain tips exist simultaneously. (Note that the keyed boxes also contain the output's scriptpubkey and amount in satoshis, but are not explicitly shown in the diagram.)

## 7. Sequential Memory Writes

The process of writing outputs to non-volatile memory can become a bottleneck. The most efficient way to write data to a storage medium such as NAND flash is sequentially, from the

last clear address forward. Our persistent trie can be implemented in a way where outputs can be added or removed using only *sequential memory writes*.

To make this concrete, consider the very simple trie built in Fig. 6. Initially, the trie contains a single output with key 1100 in memory address A (Fig. 6 i). Addresses B through G are clear. (Note: each key will be stored along with a value, but these values are omitted for clarity in the following discussion). To add output 0101 to the prefix trie, 0101 is written to the next clear memory address, which is B, and then a root node pointing to A (which contains 1100) and to B (which contains 0101) is written sequentially to C (Fig. 6 ii). The common prefix for this node is \*\*\*\*, because 1100 and 0101 do not have any prefix bits in common. The now-updated UTXO set can be referenced by the address of its root (C); however, the previous state of the UTXO set persists and can be accessed from A.

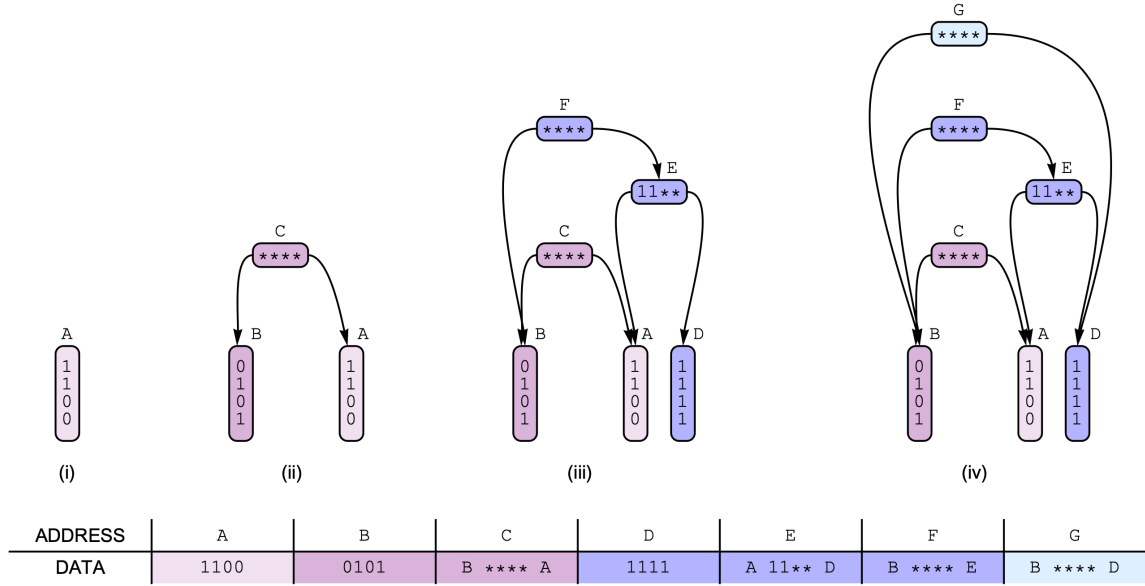


Fig. 6. Any output can be added or removed from the persistent bitwise prefix trie by writing data sequentially from the last clear address forward. Initially in (i), only 1100 exists. 0101 is added in (ii) by writing to B and C, 1111 is added in (iii) by writing to D, E, and F, and 1100 is removed in (iv) by writing to G.

To add output 1111, 1111 is written to the next free memory address, which is D (Fig. 6 iii). Then a new interior node pointing to A (which contains 1100) and to D (which contains 1111) is written to address E. The prefix for this node is 11\*\* because 1100 and 1111 both begin with two leading ones. Finally, a new root node that points to B and to E (the new interior node) is written to F. The trie has been updated by writing data sequentially: first to address D, then to E, and finally to F.

Removing an output proceeds in a similar fashion. As an example, the output 1100 can be removed by writing a new root node to address G, which skips over the node that contained 1100 (Fig. 6 iv). Indeed, whether adding or removing an output, the trie can always be updated by writing data sequentially from the last clear address in memory forward. The example hardware circuit on GitHub<sup>9</sup> can traverse through the trie at a rate of one node per clock cycle.

## 8. Defragmentation

Each time a new branch is added (to `spend` or `mint` an output), the trie stretches further over the memory space—a process we will refer to as *fragmentation*. Eventually the trie will reach the end of physical memory. The trie must be defragmented to free up space before this occurs (Fig. 7).

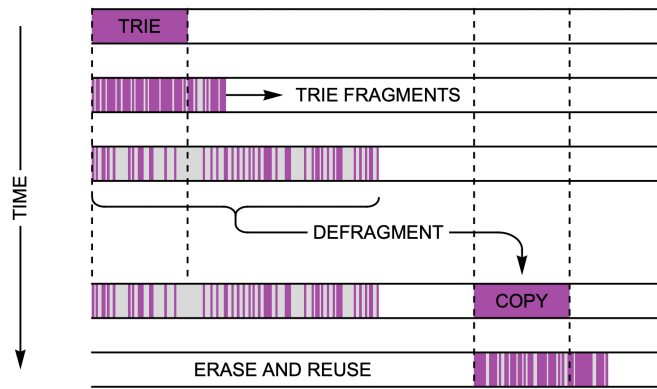


Fig. 7. Updates to the trie cause it to become fragmented across the memory space. A defragmentation routine is needed to make more room for its continued growth before it reaches the end of physical memory.

To defragment a trie, the cell’s controller traverses it, for example, from the smallest key to the largest, copying each node sequentially into a new block of memory. When copying, the controller adjusts the right and left pointers of each interior node to reflect the new physical addresses of the data. Such a defragmentation algorithm is demonstrated in the Python code available on GitHub.<sup>10</sup>

When defragmentation is complete, UTXO operations should be applied to the new defragmented copy of the trie. In order to preserve the historical UTXO contexts, the old trie should not be immediately erased. Instead it should be recycled only when a new block of memory is required. Note that reorgs to *very* old previous states remain possible at the cost of additional latency—even after those views of the UTXO set were erased—by *rewinding* the blockchain (*i.e.*, deleting the outputs that were minted and adding back the outputs that were spent).

## 9. Locality of Reference and Memory Consumption

*Locality of Reference*—One might expect that a branch of the trie, selected at random, would have its interior nodes spread out across the memory space randomly. But this is not the case: the interior nodes closer to the branch’s leaf are more likely to occupy locations near to one another (Fig. 8). This pattern is a result of writing the trie’s interior nodes in sorted order during the defragmentation routine.



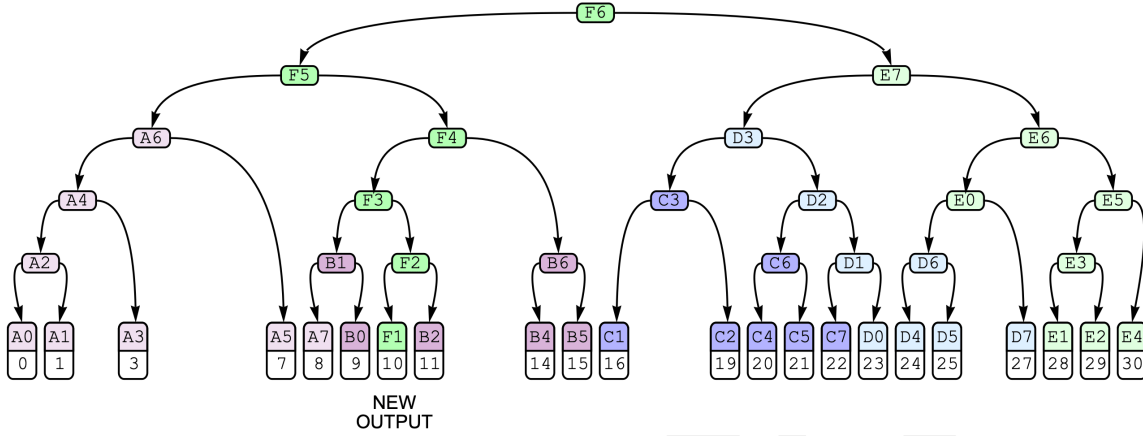


Fig. 8. Simplified illustration of a compact bitwise prefix trie one operation after defragmentation. The tree is color coded according to the page of memory in which each node exists. Observe that outputs tend to be clustered together by key. The output with a key of 10 (shaded green) was added immediately after the defragmentation event and breaks this pattern, however, the entire branch for this new output resides on page F. A NAND flash die with a 16 kB page size might hold  $10^3$  nodes on each memory page.

As new branches are written, some of these localized interior nodes will be replaced. But a new pattern emerges: each new branch will itself be localized in memory (because it too gets written sequentially). The interior nodes nearest to the root on these new branches will frequently be replaced (due to prefix collisions), but a portion of the branch towards the leaf will remain unchanged. Assuming that the base of the trie is *thick* (all available prefixes up to a certain bit-depth are filled), a new branch will update the last branch's root node with probability 1, its first interior node with probability  $\frac{1}{2}$ , and its  $m^{\text{th}}$  interior node with probability  $\frac{1}{2^m}$ . If we assume  $n$  database operations were performed after a defragmentation ( $n$  new branches were written), then a line drawn through the trie at a bit-depth of  $\log_2 n$  represents a sort of boundary: on the leaf side of this line, interior nodes will tend to be localized in memory while on the root side they will be more spread out.

*Memory Consumption*—From the discussion earlier in the manuscript, we can immediately write:

|                             |                                                   |
|-----------------------------|---------------------------------------------------|
| number of leaves            | $N$ ,                                             |
| average leaf size:          | $q_{\text{leaf}}$ ,                               |
| size of all leaves:         | $Nq_{\text{leaf}}$ ,                              |
| number of interior nodes:   | $N - 1$ ,                                         |
| average interior node size: | $q_{\text{interior}}$ ,                           |
| size of all interior nodes: | $(N - 1)q_{\text{interior}}$ ,                    |
| size of prefix trie:        | $Nq_{\text{leaf}} + (N - 1)q_{\text{interior}}$ . |

Spending an output requires traversing a trie branch from root to leaf, fetching the `scriptpubkey`, `amount`, and other details and then rebuilding the branch to skip over the now-spent coin. Minting is a similar process except the `scriptpubkey`, `amount` and other details need to be written instead of read.<sup>11</sup> In both cases, memory is consumed by the sequential writes used to rebuild the branch (*cf.* Section 8). If the average branch length is  $L$  ( $\approx \log_2 N$ ), then the average amount of memory written rebuilding the branch is  $q_{\text{interior}}L$ . If an output is minted, then a further amount  $q_{\text{leaf}}$  is written.

If  $S$  and  $M$  are the number of UTXOs spent and minted over some interval, then the number of bits written to memory over the same interval is  $(S + M)q_{\text{interior}}L + Mq_{\text{leaf}}$ . By taking the derivative of this expression with respect to time, we can write equations for the average rate at which memory is written

$$\begin{aligned} \text{due to fragmentation of interior nodes:} & \quad (\dot{S} + \dot{M})q_{\text{interior}}L \quad \text{and} \\ \text{due to appending leaves:} & \quad \dot{M}q_{\text{leaf}}. \end{aligned}$$

In the equations above,  $\dot{S} = dS/dt$  and  $\dot{M} = dM/dt$  are the bit rates at which outputs are spent and minted, respectfully.

## 10. Deferred Root Construction

Each `spend` or `mint` operation adds a new branch to the trie that terminates at a new root. From this root, the new view of the UTXO set is accessible. Historical views remain accessible from the previous roots. But if there is no need to preserve *all* historical views, then there is no need to extend all new branches down to new roots. Memory consumption can then be reduced and UTXO operations accelerated. The base of the trie can be rebuilt as necessary, for example to calculate a fingerprint for the UTXO set (*cf.* Section 12).

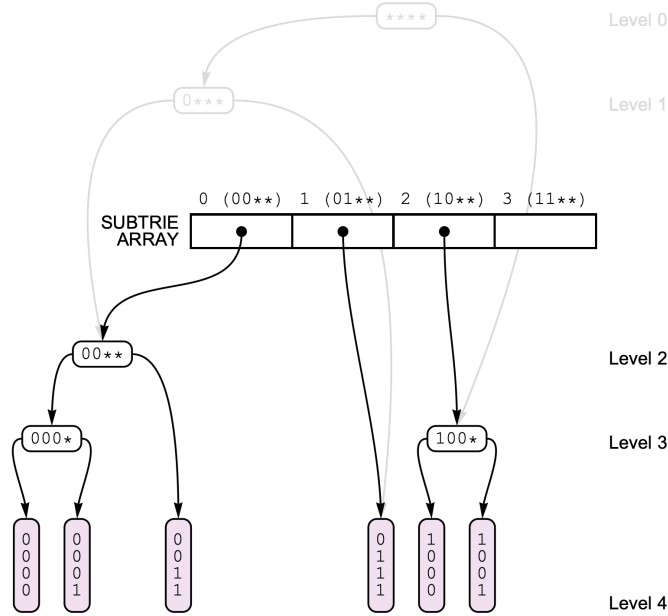


Fig. 9. Updating the first  $l$  levels of the trie can be deferred until a new root is required (e.g., a root to a view of the UTXO set corresponding to a new block). For the illustrated case of  $l = 2$ , an array of length  $2^l = 4$  stores the addresses of the roots (if they exist) of the four possible subtrees. The index of the array, expressed as a  $l$ -bit unsigned integer, implies the common prefix for the keys in the associated subtree.

Fig. 9 illustrates a suitable construction that defers updating the lowest  $l$  levels of the trie. An array of length  $2^l$  is used to store the addresses of the roots (that exist) of the  $2^l$  possible subtrees. The first  $l$  bits of a key's prefix, interpreted as an unsigned integer, specify the position in the array that holds the address of the relevant subtree's root (if it exists). The cell's controller can locate the root of any subtree with one read from the array, and then find an output in that subtree traversing approximately  $L - l$  nodes, rather than  $L$ , thereby accelerating lookups. When writing a new branch, the  $l$  nodes nearest the root do not need to be written, thereby reducing memory consumption.

[ADD SOMETHING ABOUT SUBTRIE BEING ON A SINGLE PAGE OF FLASH]

## 11. Trie Operators

An operator can be applied to our data structure, for example to add up bitcoin's money supply or to calculate a unique fingerprint for the UTXO set. Consider the compact bitwise prefix trie from Fig. 3 expressed in *functional form*:

$$((((e_0, e_1), e_3), e_7), (((e_8, e_9), e_{11}), (e_{14}, e_{15}))).$$

The symbol  $e_k$  represents the transaction output with key  $k$ , where  $k$  is an element of the keyspace  $\mathbb{K} \equiv \{0, 1, \dots, 2^K - 1\}$ . Any suitably defined binary operator can be applied to a trie expressed in this way by replacing the commas with the operator using *infix notation*. As a trivial example, consider the binary operator  $\oplus$  defined as  $x \oplus y = x+y$ ,  $e_i \oplus e_j = i+j$ ,  $x \oplus e_i = e_i \oplus x = x+i$  for  $x, y \in \mathbb{N}$  where  $\mathbb{N} \equiv \{0, 1, 2, \dots\}$  and for the transaction outputs with keys  $i, j \in \mathbb{K}$ . Applying this operator to our trie sums the numeric value of the keys:

$$((((e_0 \oplus e_1) \oplus e_3) \oplus e_7) \oplus (((e_8 \oplus e_9) \oplus e_{11}) \oplus (e_{14} \oplus e_{15}))) = 68.$$

Observe how the partial sums are calculated locally and propagated through the structure to determine the total sum, as shown in Fig. 12. When a new output is minted, for example one with a key of 12, the partial sums can be updated by applying the  $\oplus$  operator along only the new branch and not to the entire trie. There is no need to recalculate the partial sums for the parts of the trie that did not change (assuming the partial sums are saved in the interior nodes). Since the branch length scales as the logarithm of the UTXO set size, *incrementally* updating the result at the root remains feasible for large sets.

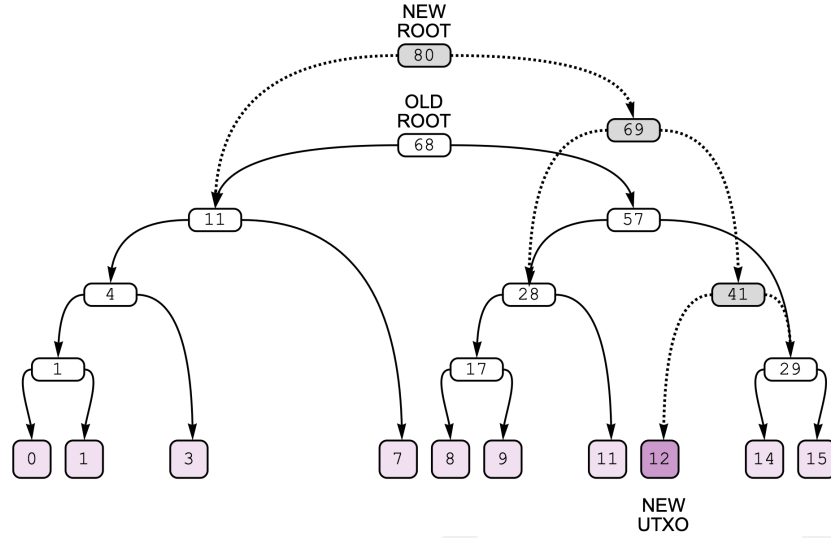


Fig. 12. A binary operator can be applied to a bitwise prefix trie. All bitcoin nodes applying the same deterministic operator to the same UTXO set will calculate the same result at the root. The illustration shows the example of addition of the keys. Observe how the partial sums can be calculated locally (and in parallel) and propagated through the structure to determine the total sum. A more realistic example would be to sum the amount field of each output to determine bitcoin’s money supply.

Since addition is both *commutative* and *associative*, the result in this example would be the same regardless of the order in which the elements were summed (the root sum could in fact be updated directly, but that misses the point of the example). But commutativity and associativity are not requirements of a trie operator. Because the trie’s shape depends only on the set of elements it contains, and not on the order in which the elements were inserted, the binary operator need only be deterministic. Any bitcoin node on the network will then calculate the same result against the same UTXO set.

## 12. Authenticated Data Structure

With only slightly more work, it would become possible for a SPV wallet to answer the question “was output  $e$  in the UTXO set as of block  $X$ ?” based on a proof received from an *untrusted* source.<sup>12</sup> To make this possible, our data structure must be *authenticated*, for example using a method similar<sup>13</sup> to Anagnostopoulos *et al.*<sup>14</sup>

We define the UTXO set’s *fingerprint*,  $\Xi$ , as the result of applying a secure binary hash operator to the trie. Applied to the example from last section, we would have:

$$\text{hash}(\text{hash}(\text{hash}(\text{hash}(e_0, e_1), e_3), e_7), \text{hash}(\text{hash}(\text{hash}(e_8, e_9), e_{11}), \text{hash}(e_{14}, e_{15}))) = \Xi.$$

But unlike the case of Anagnostopoulos where the fingerprint was digitally signed by a trusted source, ours would be authenticated by proof-of-work. This would require a new consensus rule enforcing an accurate commitment to the UTXO fingerprint in each new block.

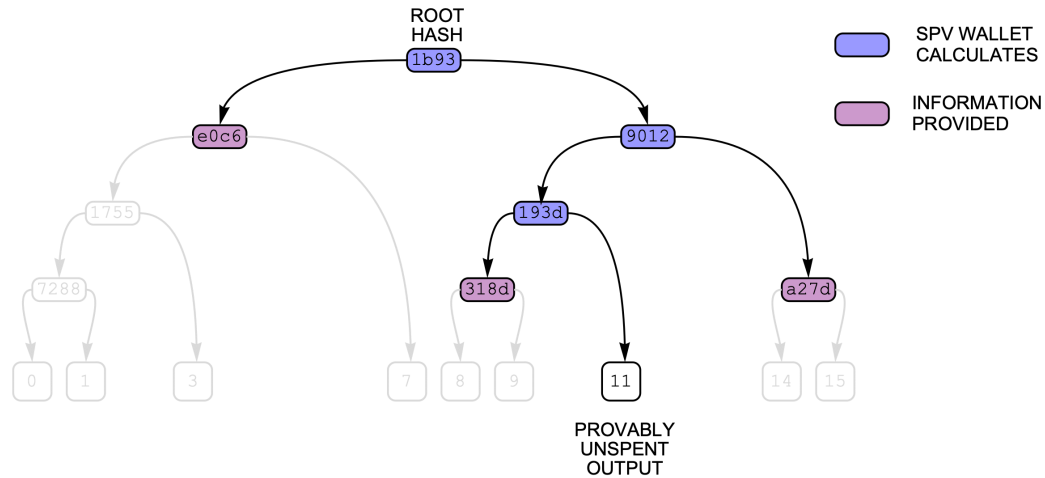


Fig. 13. If the miners commit the root hash for the UTXO set in each block then it becomes possible to prove that an output is unspent in a compact way. Proof of absence is also possible by providing two inclusion proofs of neighboring keys, demonstrating the absence of the key in question. This would allow SPV wallets to validate unconfirmed transactions.

A proof that an output is a member of the UTXO set as of block  $X$  then consists of the interior hashes shaded purple in Fig. 13, and whether the hash is right or left sided. The SPV wallet would use the provided information to calculate a local version of the fingerprint. The proof is valid if that local version matches the authenticated fingerprint. A proof that an output is *not* a member of the UTXO set consists of providing two inclusion proofs: one for the output with the smallest key that is larger than the output in question, and another for the output with the largest key that is smaller than the output in question. For example, if the SPV wallet were given inclusion proofs for outputs 9 and 11, then it would also possess proof that no output with a key of 10 is a member of the set. Combined with standard Merkle branch proofs, this would give SPV wallets the ability to validate unconfirmed bitcoin transactions.

A second advantage is that new bitcoin nodes could be brought online quickly by downloading a recent copy of the UTXO set from an *untrusted* source (3.6 GB for BTC at the time of writing). The node would assemble the set into a compact bitwise prefix trie, apply the hash operator, and then compare the result with the authenticated fingerprint. If the two match, then the downloaded UTXO set could be assumed accurate and complete. This would save new nodes from having to download the entire blockchain (250 GB for BTC at the time of writing) in order to build the UTXO set.

*Hashing is faster than signature validation*—Using our method, the fingerprint can be updated incrementally with logarithmic complexity in the set size. It requires hashing  $l$  chunks of size  $2\Delta q$  bytes per UTXO operation, where  $l$  is the branch length and  $\Delta q$  is the size of each hash digest. Typical values might be  $\Delta q = 256$  bits and, at global scale,  $l = 40$ . If construction of the base of the trie is deferred following the method of Section 10 then the number of bytes hashed can be further reduced. Napkin math<sup>15</sup> suggests that verifying a transaction’s signature is probably over two orders of magnitude more costly than this amount of hashing, given efficient hardware implementations for both hashing and for scalar multiplication of elliptic curve points. The time it takes to update the fingerprint would not be the bottleneck, although

nodes would be forced to maintain their UTXO databases in a normative fashion (*i.e.*, using a compact bitwise prefix trie). The method of rolling UTXO set hashes<sup>16</sup> does not have this constraint, although the rolling method does not permit compact inclusion proofs.

*Validationless mining would become more costly*—What is perhaps more interesting is that in order for a mining node to create a new block candidate, it must calculate the new UTXO fingerprint for the candidate block. Even if the miner mines an “empty block,” that block’s coinbase transaction will still alter the UTXO fingerprint from the previous block. This would have a significant impact on so-called “validationless mining”—the practice of mining a new block on top of a block header before downloading and verifying the transactions in the corresponding block. In order to calculate the new fingerprint the node would have to first update its UTXO set to a view consistent with the block upon which it is building, which, as a practical matter, means downloading and processing the previous block. Furthermore, nodes would be forced to maintain their UTXO databases in a normative fashion (*i.e.*, using a compact bitwise prefix trie). The method of rolling UTXO set hashes<sup>17</sup> does not have this constraint, although the rolling method does not permit compact inclusion proofs.

*A few words on implementation*—The “straightforward” way to commit to the fingerprint would be to write the fingerprint to a new field added to the block header for this purpose. In practice, this would not be straightforward at all because it would break mining infrastructure and SPV wallets. The method of adding the commitment that would be least invasive might be to add the commitment to the block’s coinbase transaction. However, this adds a slight wrinkle: the UTXO fingerprint for a given block depends on all the transactions in that block—including the coinbase transaction—but then adding the fingerprint to the coinbase transaction would change the coinbase transaction hash and thus its location in the trie and result in a different fingerprint. This problem can be avoided if it becomes customary to committing to the UTXO fingerprint before adding the coinbase transaction to the UTXO set. When the block is connected at tip, then the UTXO fingerprint will change when the coinbase is added. This has no real impact on the usefulness of the commitment because coinbase transactions must age for 100 blocks anyways.

### 13. Related Work

In terms of using an *authenticated data structure* to maintain the UTXO set, possibly the earliest related efforts can be found in the 2012 Bitcointalk thread “Ultimate Blockchain Compression” [ref]. In this thread, Andrew Miller builds a proof of concept of an authenticatable data structure for the UTXO set based on the red-black tree [ref], clearly inspired by the Anagnostopoulos paper. Alan Reiner observes that its shape is not deterministic, leading to the undesirable property that information in addition to the set of unspent outputs is required to calculate the fingerprint [ref]. In the same thread, TierNolan and d’Aniel discuss binary prefix tries in the context of UTXO set, although neither appear to pursue the idea in any depth.

In terms of using a *persistent data structure* to yield native parallel validation and instant blockchain reorganizations, the earliest work the author is aware of is the BitCrustDB by Tomas van der Wansem [ref]. BitCrust uses a novel “spend tree” rather than a UTXO set, which results in some interesting properties. Tom Zander’s “Flowee the Hub” and Joannes Vermorel’s prototype “CashDB” both use persistent data structures for managing the UTXO set while

Awemany’s “Storm” prototype used a persistent data structure to manage weak-block chain tips [ref].

The benefits of parallelization (*e.g.*, by prefix) and horizontal scaling in hardware are obvious. Despite this, the UTXO database for Bitcoin Core permits only serial access and requires locks, eleven years after the inception of bitcoin.

An article by Joannes Vermorel<sup>18</sup> *et al.* in 2016?? proposes using a “merlix tree” for organizing the transactions in a block rather than maintaining the UTXO set, and points out that the structure is deterministic (it has the property that Reiner was looking for that Miller’s red-black tree did not possess). The “merklix tree” described in Vermorel’s article is similar to the compact bitwise prefix trie presented in this article, but different in a way that would make its implementation in hardware less efficient.<sup>19</sup>

(iii) `spend` and `mint` operations with logarithmic complexity,<sup>20</sup> Earlier work assumed that updates to the fingerprint would be too slow and that the commitment to the UTXO state must be lagging.<sup>21</sup>

## 14. Conclusion

We have proposed a binary-tree-based data structure for maintaining Bitcoin’s UTXO set that is parallelizable, persistent and authenticated by proof-of-work. The structure is scalable and suitable for parallelization across different pieces of physical hardware. On GitHub we provided an example hardware circuit that can navigate through the tree at a rate of one node per clock cycle. Adding or removing outputs from the UTXO set can be performed with only sequential memory writes, leaving historical views of the set intact. This property then allows mining nodes to natively process competing blocks in parallel or instantly switch to a previous view of the UTXO set upon a reorganization of the blockchain. Authentication is achieved by mining nodes committing a fingerprint of the current UTXO set for each block, the accuracy of the fingerprint would be enforced by a new protocol rule. The fingerprint can be updated incrementally as outputs are spent and minted and with a cost small in comparison to digital signature verification. New network nodes can use this fingerprint to confirm that a UTXO set downloaded from untrusted sources are accurate and complete, saving them from downloading and rebuilding the UTXO set from the much-larger blockchain. Users’ mobile wallets can also use the authenticated fingerprints to verify unconfirmed bitcoin transactions from a compact proof received from a network node, rather than maintaining the complete UTXO set on low-power/low-bandwidth device.

## References

<sup>1</sup> Gavin’s UTXO oopsie

<sup>2</sup> Cite “steps to run the network” from the white paper.

<sup>3</sup> Note that the transaction ID XORed with the output index is a unique 256-bit identifier that could be used directly as a key. Such keys would not be randomly distributed over the keyspace however.

<sup>4</sup> Originally, the only additional information needed was the boolean `is_coinbase` and, in the case where that boolean was true, the height of the block into which the new coinbase output was mined, `mined_height`. This information was needed to enforce the rule that new coinbase outputs must age for 100 blocks before being spent. With the introduction of `relative_locktime` and `checkSequenceVerify`, the applicability of `mined_height` expanded to non-coinbase transactions as well. Today, an output can be encumbered in a way that prevents a transaction from spending it until after a number of blocks *relative to* the block height where it was mined. For each *key* in today's UTXO set, the *value* is the tuple (`scriptpubkey`, `amount`, `is_coinbase`, `mined_height`).

<sup>5</sup> An individual transaction or a block must be committed *atomically* by an external process and no output can be spent twice. But there is no need for locks: when verifying a group of transactions, each transaction's inputs can be speculatively spent. Should the spending of any input in a given transaction fail (*e.g.*, it was already spent by a concurrent process), then all the speculatively spent inputs for the failed transaction must be "unspent" and the transaction rejected or moved to a retry queue. Atomicity for a block can be ensured, for example, by using Tomas van der Wansem's "outputs then inputs" algorithm: all of the outputs are added to the UTXO set and then all of the inputs are removed. If any step fails, the block is rejected and the changes unwound. Unwinding the changes is trivial because the data structure is persistent.

<sup>6</sup> Pedantically speaking, the worst case is linear scaling, for example if the tree collapsed to a linear linked list. This is not a concern in our application.

<sup>7</sup> Bitcoin Unlimited, a software fork of Bitcoin Core, was the first full node implementation with the ability to validate multiple blocks in parallel. Parallel validation was merged on 2 March 2017:  
<https://github.com/BitcoinUnlimited/BitcoinUnlimited/pull/254>

<sup>8</sup> Driscoll J. R., Sarnak, N., Sleator, D. D., Tarjan, R. E "Making data structures persistent." In *Proceedings of the Eighteenth Annual ACM Symposium on Theory of Computing*. p. 109–121. (1986)  
doi:10.1145/12130.12142

## <sup>9</sup> Github verilog

<sup>10</sup> A Cashdrive simulator written in Python along with a detailed readme file is available from  
<https://github.com/1PeterR/cashdrive>

<sup>11</sup> An observant reader may have noticed that for `mint` operations, it is possible in some cases to infer the location of the new leaf node directly from the prefixes of the interior nodes, potentially saving a random read from flash.

<sup>12</sup> Coin supply could be committed as well.

<sup>13</sup> We use the compact persistent bitwise prefix trie rather than the persistent red-black tree considered by Anagnostopoulos *et al.*



<sup>14</sup> Anagnostopoulos, A., Goodrich, M., Tamassia, R. “Persistent Authenticated Dictionaries and Their Applications.” In *Proceedings of the 4th International Conference on Information Security*, p. 379-393 (2001) <http://cs.brown.edu/cgc/stms/papers/isc2001.pdf>

<sup>15</sup> Mark Lundberg estimated that hashing is >10,000 times more efficient than elliptic curve point scalar multiplication in an apples-to-apples contest between efficient hardware implementations of each function: <https://gist.github.com/markblundeberg/94650e69ebf056215de6ad1a716de559#collision-attacks>

<sup>16</sup> Wuille, P. “Rolling UTXO set hashes.” *Bitcoin Developer Mailing List*, 15 May 2017  
<https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2017-May/014337.html>

<sup>17</sup> Wuille, P. “Rolling UTXO set hashes.” *Bitcoin Developer Mailing List*, 15 May 2017  
<https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2017-May/014337.html>

<sup>18</sup> Vermorel, J., Séchet, A., Chancellor, S., Cox, J. "Merklix Tree for Bitcoin." No publisher (2018)  
<https://blog.vermorel.com/pdf/merklix-tree-for-bitcoin-2018-07.pdf>

<sup>19</sup> The Merkl tree is not *compact*: it does not have the property that the number of interior nodes is exactly one fewer than the number of elements in the set.

<sup>20</sup> An attacker can increase the search complexity only marginally, as grinding deep prefix collisions is costly and mostly ineffective (in the worst case, the branch length increases to the bit length of the key space).

<sup>21</sup> This discussion from 2015 in the Bitcoin Wizard’s IRC channel shows the many problems that needed to be solved to build an authenticated UTXO commitment scheme. At the time of this discussion, it was assumed that commitments would need to be lagging by at least one block. <http://gnusha.org/bitcoin-wizards/2015-09-18.log>